Statistics

```
It is the study of collection, organization, displaying, analysing,
interpretation, and presentation of data
```

Numpy

```
it provides high performance multidimesional array object and tools to work
with array
```

Importing Numpy

```
import numpy as np
np.__version
dir(np) #check available methods
```

Creating Numpy Array

```
#int array
numpy_array = np.array(list) # tuple also can
#float array
array = np.array(list, dtype="float")
#dtype can be int, np.commplex128
#boolean array
array = np.array(list, dtype=bool)
```

Multi Dimensional Array

```
#get row
array[index]
#get column
array[:, index]
#slicing
array[index:index, index:index]
    # row, column
#reverse
array[::-1, ::-1]
```

Array to List

```
list = numpy_array_from_list.tolist()
```

Shape and Size of an Array

```
shape = array.shape
# (row, column)
size = array.size
# total item
```

Mathematical Operation with Numpy

```
can do all operations I know
```

Checking Data Types

```
array.dtype
```

Converting Data Types in Array

```
array_name.astype("int")
#str, ...
```

Create Array of Zeros and Ones

```
array_name = np.zeros(shape, dtype=, order"C/F")
# C-order is based on row
# Fortran order is based on column
```

Reshape

```
array_name.reshape(shape)
```

Flatten

```
flattened = array_name.flatten()
```

Stack

```
#horizontal stack
np.hstack((np_one, np_two))

#vertical stack
np.vstack((np_one, np_two))
```

Generate Random Number

| Random Float

```
random_number = np.random.random(numberstogenerate)
#OR
random_number = np.random.rand(shapeoptional)
```

| Random Int

```
random_number = np.random.randint(start, end, size=shape/size) #size is
optional
```

| Random Number from Normal Distribution

```
number = np.random.normal(mu, sigma, size)
# mu is the mean, sigma is SD
#OR
number = np.random.randn(shape)
#meannya 0
```

| Random Choice

```
number = np.random.choice([listofchoices], size=optional)
```

Mathplot and Seaborn

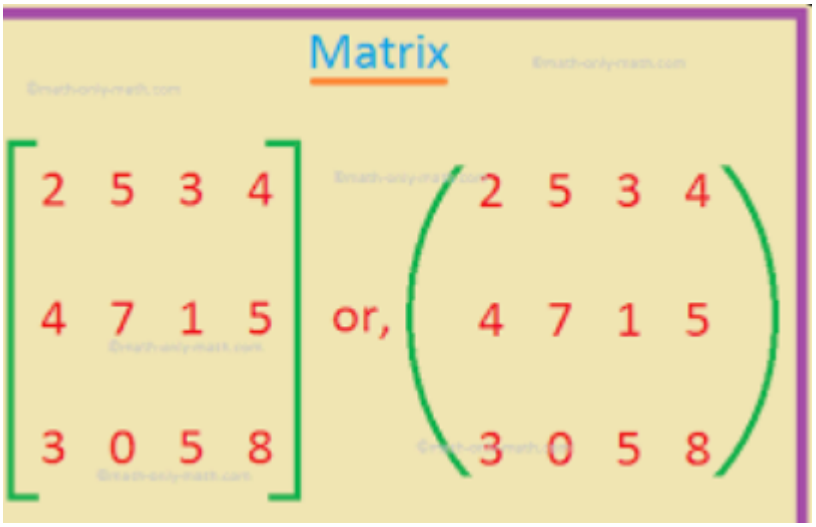
```
import mathplotlib.pyplot as plt
# to create graph
import seaborn as sns
# design graph better


sns.self()
plt.hist(array_name, color="barcolor", bins=50)
#bins is how many group of bars

plt.show()
plt.plot(x, y)
plt.xlabel("")
plt.ylabel("")
plt.title("")
plt.xticks() #sumbu x apa aja
```

Matrix in Numpy

```
np.matrix(array_name, dtype=)
```



| Modify Matrix

```
np.asarray(matrix_name)[index] = value
#np.asarray means to treat matrix as an array
```

| Matrix Multiplication

```
np.matmul(matrix1, matrix2)
```

| Matrix Determinant

```
np.linealg.det(matrix)
```

Numpy Arange

```
like range()
```

```
np.arange(start, stop, step)
```

Sequence of Number by Linspace and Logspace

| Linspace

```
evenly spaced number by + or -
```

```
np.linspace(start, end, num=howmanynum, endpoint=True/False)
# endpoint include end or no
```

| Logspace

```
the space is multiplied or divided
```

```
np.logspace(start, stop, num=, endpoint=)
```

Numpy Statistical Functions

```
array_name.min()
array_name.max()
array_name.mean()
array_name.median()
array_name.var()
array_name.percentile(array, whichpercentile)

#OR

#applied to every functions
np.amin(array, axis=0/1)
#axis 0 operation per column, 1 operation per row
```

Create Repeating Sequences

```
np.tile(array, times)
# 123 123
np.repeated(array, times)
# 11 22 33
```

Linear Algebra Dot Product

```
array1 = [1, 2, 3]
array2 = [4, 5, 6]
np.dot(array1, array2)
# 1x4 + 2x5, 3x6
```